

```
        validMoves.push(RIGHT);
    }
    return validMoves;
}
```

The first thing the `getValidMoves()` function does is call `findBlankSpace()` and store the x, y coordinates of the blank space in the variables `blankx` and `blanky`. Next, the function sets up the `validMoves` variable with an empty Python list or empty JavaScript array to hold all the valid directions for a slide.

Looking back at [Figure 12-5](#), a y-coordinate of 0 represents the top edge of the board. If `blanky`, the blank space's y-coordinate, is not 0, then we know the blank space is not on the top edge. If the previous move was also not DOWN, then `up` is a valid move, and the code adds `UP` to `validMoves`.

Similarly, the left edge has an x-coordinate of 0, the bottom edge has a y-coordinate of `SIZE - 1`, and the right edge has an x-coordinate of `SIZE - 1`. Using the expression `SIZE - 1` ensures that this code works no matter whether the board is 3×3 , 4×4 , or any other size. The `getValidMoves()` function does these checks for all four directions and then returns `validMoves`.

Next, the `getNewPuzzle()` function returns the data structure of a scrambled board for the program to solve. Tiles can't simply be randomly placed on the board, because some configurations of tiles produce puzzles that are impossible to solve. To avoid this, the `getNewPuzzle()` function starts with an ordered, solved board and then applies a large number of random slides to it. Solving this puzzle is, in effect, figuring out the moves that undo these slides. The Python code for the `getNewPuzzle()` function is as follows:

Python

```
def getNewPuzzle():
    """Get a new puzzle by making random slides from the solved state."""
    board = getNewBoard()
    for i in range(DIFFICULTY):
        validMoves = getValidMoves(board)
        makeMove(board, random.choice(validMoves))
    return board
```

The JavaScript code is as follows: